Practical 1. Implement KNN Classification to Predict Whether a Student Will Pass or Fail Based on Study Hours and Attendance.

```
import pandas as pd
from sklearn.model_selection import train_test_split
from sklearn.preprocessing import StandardScaler
from sklearn.neighbors import KNeighborsClassifier
from sklearn.metrics import accuracy_score, classification_report

df = pd.read_csv("student_result.csv")

X = df[["StudyHours", "Attendance"]]
y = df["Result"]

X_train, X_test, y_train, y_test = train_test_split(
    X, y, test_size=0.25, random_state=42, stratify=y
)

scaler = StandardScaler()
X_train = scaler.fit_transform(X_train)
X_test = scaler.transform(X_test)

model = KNeighborsClassifier(n_neighbors=3)
model.fit(X_train, y_train)

y_pred = model.predict(X_test)

print("Actual :", list(y_test))
print("Predicted:", list(y_pred))
print("Accuracy:", accuracy_score(y_test, y_pred))

new_student = scaler.transform([[4.0, 76]])
print("New Student Prediction:", model.predict(new_student)[0])
```

Vansh Prajapati

Result:

The program displays the actual and predicted classes, test accuracy, and the prediction for a new student with 4 study hours and 76% attendance. With this dataset, the new student is expected to be classified as Pass.

Practical 2. Implement KNN Classification to Predict Whether a Customer Will Buy a Product Based on Age and Monthly Income.

```
import pandas as pd
from sklearn.model_selection import train_test_split
from sklearn.preprocessing import StandardScaler
from sklearn.neighbors import KNeighborsClassifier
from sklearn.metrics import accuracy_score

df = pd.read_csv("customer_purchase.csv")

X = df[["Age", "MonthlyIncome"]]
y = df["Purchased"]

X_train, X_test, y_train, y_test = train_test_split(
    X, y, test_size=0.25, random_state=42, stratify=y
)

scaler = StandardScaler()
X_train = scaler.fit_transform(X_train)
X_test = scaler.transform(X_test)

model = KNeighborsClassifier(n_neighbors=3)
model.fit(X_train, y_train)

pred = model.predict(X_test)

print("Actual   :", list(y_test))
print("Predicted:", list(pred))
print("Accuracy :", accuracy_score(y_test, pred))

new_customer = scaler.transform([[33, 37000]])
print("New Customer Prediction:", model.predict(new_customer)[0])
```

Vansh Prajapati

Result:

The program reports the test-set accuracy and predicts the purchase decision of a new customer. For the given example (age 33 and income ₹37,000), the model is expected to predict Yes.

Practical 3. Compare Different Values of K in KNN Classification and Find the Effect of K on Accuracy.

```
import pandas as pd
from sklearn.model_selection import train_test_split
from sklearn.preprocessing import StandardScaler
from sklearn.neighbors import KNeighborsClassifier
from sklearn.metrics import accuracy_score

df = pd.read_csv("k_comparison.csv")

X = df[["Hours", "Attendance"]]
y = df["Result"]

X_train, X_test, y_train, y_test = train_test_split(
    X, y, test_size=0.25, random_state=42, stratify=y
)

scaler = StandardScaler()
X_train = scaler.fit_transform(X_train)
X_test = scaler.transform(X_test)

for k in [1, 3, 5, 7]:
    model = KNeighborsClassifier(n_neighbors=k)
    model.fit(X_train, y_train)
    pred = model.predict(X_test)
    print("K =", k, "Accuracy =", accuracy_score(y_test, pred))
```

Vansh Prajapati

Result:

The output prints one accuracy value for each K: 1, 3, 5, and 7. Students should observe how changing K changes the prediction accuracy. The best K for this particular train/test split is the one producing the highest displayed accuracy; no single K is universally best.

Practical 4. Implement KNN Regression to Predict House Price Using Area and Number of Bedrooms.

```
import pandas as pd
from sklearn.model_selection import train_test_split
from sklearn.preprocessing import StandardScaler
from sklearn.neighbors import KNeighborsRegressor
from sklearn.metrics import mean_absolute_error, r2_score

df = pd.read_csv("house_price.csv")

X = df[["Area", "Bedrooms"]]
y = df["PriceLakh"]

X_train, X_test, y_train, y_test = train_test_split(
    X, y, test_size=0.25, random_state=42
)

scaler = StandardScaler()
X_train = scaler.fit_transform(X_train)
X_test = scaler.transform(X_test)

model = KNeighborsRegressor(n_neighbors=3)
model.fit(X_train, y_train)

pred = model.predict(X_test)

print("Actual prices   :", list(y_test.round(2)))
print("Predicted prices:", list(pred.round(2)))
print("MAE:", mean_absolute_error(y_test, pred))
print("R2 Score:", r2_score(y_test, pred))

new_house = scaler.transform([[1100, 3]])
print("Predicted price for new house:",
      round(model.predict(new_house)[0], 2), "Lakh")
```

Vansh Prajapati

Practical 5.Use KNN Classification to Identify a Flower Species Using Sepal and Petal Measurements.

```
import pandas as pd
from sklearn.model_selection import train_test_split
from sklearn.preprocessing import StandardScaler
from sklearn.neighbors import KNeighborsClassifier
from sklearn.metrics import accuracy_score, confusion_matrix

df = pd.read_csv("flower_data.csv")

features = ["SepalLength", "SepalWidth", "PetalLength", "PetalWidth"]
X = df[features]
y = df["Species"]

X_train, X_test, y_train, y_test = train_test_split(
    X, y, test_size=0.30, random_state=42, stratify=y
)

scaler = StandardScaler()
X_train = scaler.fit_transform(X_train)
X_test = scaler.transform(X_test)

model = KNeighborsClassifier(n_neighbors=3)
model.fit(X_train, y_train)

pred = model.predict(X_test)

print("Actual   :", list(y_test))
print("Predicted:", list(pred))
print("Accuracy :", accuracy_score(y_test, pred))
print("Confusion Matrix:")
print(confusion_matrix(y_test, pred))

new_flower = scaler.transform([[6.0, 3.0, 4.5, 1.5]])
print("New Flower Prediction:", model.predict(new_flower)[0])
```

Vansh Prajapati

Result:

The program displays actual and predicted flower species, classification accuracy, a confusion matrix, and the predicted species for a new flower. The sample new flower is expected to be classified near the Versicolor group.